

Google Antigravity 對話紀錄

對話 ID: 1862ef5e-0192-4cc7-ac90-f3aa9fb369f7 | 專案名稱: Ticket Merge

U

使用者 (User)

skills和workflows的差異?

A

Antigravity (Assistant)

在 Google Antigravity 生態系中，**Rules** (規則)、**Workflows** (工作流程) 和 **Skills** (技能) 是用來引導和自訂 Agent 行為的三個不同層級。

以下是它們的核心差異與對比：

特性	Rules (規則)	Workflows (工作流程)	Skills (技能)
角色比喻	員工守則 / DNA	SOP 檢查清單	專業工具箱
主要目的	定義 Agent 的基本身份、行為約束與全局偏好。	定義結構化、多步驟的固定執行程序。	擴充 Agent 在特定領域的專業能力與知識。
觸發方式	被動且全局：設定後在所有對話中自動且持續生效，無需特別觸發。	主動且明確：通常由使用者輸入特定斜線指令 (如 <code>/weekly-report</code>) 手動觸發。	自動且情境相關：Agent 偵測到任務需要時，會自動讀取並調用 (漸進式揭露)。
常見範例	「預設使用繁體中文」、「使用 uv 建立 Python 虛擬環境」。	收集本週程式碼變更 $\rightarrow$ 自動撰寫摘要 $\rightarrow$ 格式化為 Markdown 報告。	台灣高鐵交易紀錄 PDF 處理技能 (包含 <code>SKILL.md</code> 指引與相關處理腳本)。
模組化	全局持久 (記錄於 <code>AGENTS.md</code>)。	針對特定任務流程設計。	高度模組化，可跨專案複製、共享或刪除。

總結

- Rules** 決定 Agent 的「行為準則」(怎麼說話、遵守什麼開發規範)。
- Workflows** 決定 Agent 的「執行步驟」(依照固定 SOP 完成特定日常任務)。
- Skills** 決定 Agent 的「專業能力」(在需要時才打開工具箱，讀取特定知識或執行腳本來解決問題)。

U

使用者 (User)

- 規範定義 (Rules)： `AGENTS.md`
- 需求對齊 (Alignment)：互動問答收斂需求
- 計畫制定 (Plan)： `implementation_plan.md`
- 進度追蹤 (Task)： `task.md`
- 平行執行 (Execution)：背景任務 & Subagents
- 驗證交付 (Verification)： `walkthrough.md`
- 沉澱進化 (Evolution)：Custom Skills & `/learn` 以上步驟適合變成workdlows嗎?

A

Antigravity (Assistant)

這套步驟非常適合整理成一個完整的 **Workflow (工作流程)** 。

這實際上是一個非常完整且嚴謹的「**Agent 協同開發生命週期 (Agentic Software Development Lifecycle)**」。它具備明確的階段性、標準產出 (Artifacts)，以及人機協作的檢查點 (Gatekeeper)，完全符合 Workflow 作為「SOP 檢查清單」的定義。

如果將這套步驟在 Antigravity 中落地，建議的架構與實作方式如下：

如何將這套步驟轉化為 Workflow 與系統設定？

這套流程不需要全部寫在同一個地方，而是可以拆解並與 **Rules**、**Workflows**、**Skills** 進行有機結合：

1. 骨幹：定義一個主工作流程 (Workflow)

您可以設計一個如 `/develop` 或 `/new-feature` 的工作流程，引導 Agent 依序執行步驟 2 至 6：

- 步驟 2：需求對齊 $\rightarrow$ 自動推薦或執行 `/grill-me` 進行互動式問答。
- 步驟 3：計畫制定 $\rightarrow$ 自動在專案目錄建立 `implementation_plan.md`，並暫停等待您的 Review 與批准。
- 步驟 4：進度追蹤 $\rightarrow$ 收到您的核可後，自動建立並初始化 `task.md` 的 TODO 列表。
- 步驟 5：平行執行 $\rightarrow$ Agent 根據 `task.md` 的規劃，自動開啟背景任務 (Background Tasks) 或派發 Subagents 進行平行開發。
- 步驟 6：驗證交付 $\rightarrow$ 執行測試並自動產出 `walkthrough.md` 報告。

2. 約束：寫入 Rules (AGENTS.md)

為了確保 Agent 每次收到新任務時都「主動」採用這套流程，您應該在 **AGENTS.md** (或全域的 Rules) 中加入以下規範：

```
# 開發流程規範
- 對於任何非微調 ( Trivial ) 的複雜開發需求，必須嚴格遵守「協同開發生命週期」：
  1. 先與使用者對齊需求。
  2. 撰寫 `implementation_plan.md` 並取得使用者核可 ( Planning Mode )。
  3. 建立 `task.md` 並在執行過程中隨時更新進度。
  4. 完成後撰寫 `walkthrough.md` 並執行驗證。
```

3. 工具：調用 Skills (技能) 來輔助執行

在 Workflow 的執行過程中，Agent 可以調用現有的技能來加速流程：
* 在步驟 5 (平行執行)：調用特定技術領域的技能 (如 `pymol`、`uniprot-database` 或您自訂的 `taiwan-hsr-ticket-processor`)。
* 在步驟 7 (沉澱進化)：當任務完成時，Agent 可以調用系統內建的 `workflow-skill-creator` 技能，將這次的特定工作成果打包，或引導您使用 `/learn` 讓 Agent 記住這個模式。

這樣做的好處

1. **可預測性高**：不論任務大小，Agent 都會按照相同的 SOP 執行，您能清楚知道目前進行到哪一步 (看 `task.md` 即可)。
2. **安全控制 (Gatekeeping)**：在「計畫制定 (Plan)」與「執行 (Execution)」之間有明確的審查機制，避免 Agent 在沒有對齊需求的情況下盲目修改程式碼。
3. **持續進化**：最後一步的「沉澱進化」確保了專案經驗能轉化為長期資產，越用越聰明。

U

使用者 (User)

如何定義主工作流程？



Antigravity (Assistant)

在 Google Antigravity 中，定義主工作流程 (Workflow) 非常直覺。工作流程本質上是一個存放在特定目錄的 Markdown (.md) 檔案 `develop.md` 對應 `/develop`)。

以下是定義主工作流程的具體步驟、結構與實戰範本：

1. 檔案存放位置

在專案根目錄下，將工作流程檔案放置於：`.agents/workflows/` (若 `workflows` 資料夾不存在，可以直接建立)

2. 檔案結構與語法

一個標準的 Workflow 檔案包含兩個部分：**1. YAML Frontmatter**：位於最上方，用 `---` 包裹，其中 `description` 欄位為必填，讓 AI 3. 格式列出具體的執行指令或指引。

💡 執行控制標記 (Annotations)：

- `// turbo`：加在步驟指令的下一行，允許 Agent 不經詢問直接自動執行該步驟。
- `// turbo-all`：寫在檔案內任何地方，代表整份工作流程的所有步驟都自動執行。

3. 您的「開發生命週期」實戰範本

以下是為您的 7 步流程設計的 `develop.md` 範本。您可以直接將此內容存為 [develop.md](#)：

```
---
description: "啟動標準的 Agent 協同開發生命週期（包含需求對齊、計畫審查、任務追蹤、自動執行與驗證）"
---

1. 需求對齊 (Alignment)
  引導使用者進行互動式問答以收斂與釐清開發需求。你可以建議使用者使用 `/grill-me` 來對齊設計決策。

2. 計畫制定 (Plan)
  根據對齊的需求，在專案根目錄建立或更新 `implementation_plan.md`。詳細說明修改範圍、影響評估與驗證計畫。
  > [!IMPORTANT]
  > 建立計畫後，請停止並等待使用者核可。在使用者批准前，嚴禁修改任何原始碼或執行破壞性指令。

3. 進度追蹤 (Task)
  計畫獲得使用者核可後，建立 `task.md`。將開發任務拆解為具體的 TODO 列表，並在後續執行時隨時更新進度狀態（如 `[ ]`，`[/]`，`[x]`）。

4. 平行執行 (Execution)
  依照 `task.md` 的規劃執行開發。如遇到耗時的編譯、測試或需要獨立研究的事項，可使用 `define_subagent` 與 `invoke_subagent` 來委派任務。

5. 驗證交付 (Verification)
  開發完成後，依據 `implementation_plan.md` 的驗證計畫進行測試。通過後在專案根目錄建立 `walkthrough.md`，說明本次變更的具體使用方式。

6. 沉澱進化 (Evolution)
  詢問使用者是否需要將此次開發的特定 Workflow 或工具提煉為自訂技能 (Custom Skill)。若是，可調用 `workflow-skill-creator` 進行提煉。
```

4. 如何觸發與執行

1. **儲存檔案**：將上述內容儲存至 `.agents/workflows/develop.md`。
2. **觸發指令**：在對話框中輸入 `/develop`，Agent 就會被喚醒，並嚴格遵循這 6 個步驟引導您完成開發。

請問您需要我直接為您在專案的 `.agents/workflows/` 目錄下建立這個 `develop.md` 檔案嗎？

U

使用者 (User)

我的想法是 引導學生開發軟體的步驟，目前有 1. 問題描述與需求分析 requirements.md 再配合以下方式，1. 規範定義 (Rules)：AGENTS.md 2. 需求對齊 (Alignment)：互動問答收斂需求 3. 計畫制定 (Plan)：implementation_plan.md 4. 進度追蹤 (Task)：task.md 5. 平行執行 (Execution)：背景任務 & Subagents 6. 驗證交付 (Verification)：walkthrough.md 7. 沉澱進化 (Evolution)：Custom Skills & /!learn 我要怎麼設計軟體開發步驟的workflows?

針對「引導學生開發軟體」的教學情境，最理想的設計是將這套流程轉化為「引導式教學工作流程 (Guided Teaching Workflows)」。

因為學生的學習重點在於思考、規劃與驗證，而非讓 AI 直接把所有程式碼寫完。因此，設計的關鍵在於「Agent 作為助教與引導者，學

我們可以用「一個 Rules 規範 (引導原則)」+「三個階段性 Workflows (SOP 指令)」來設計這套軟體開發步驟。

第一步：定義引導規則 — AGENTS.md (Rules)

在專案根目錄的 `.agents/AGENTS.md` 中，定義 Agent 的「助教身份行為準則」。這能確保 Agent 在執行任何步驟時，都不會直接「給

```
# 學生開發教學引導原則

- **角色定位**：你是一位有耐心的資深軟體開發助教。請使用繁體中文。
- **引導原則**：
  1. **不要直接給出完整的程式碼**：提供結構範例、虛擬碼 (Pseudo-code) 或關鍵 API 提示，引導學生自己動手寫。
  2. **引導學生思考而非給予解答**：如果程式出錯，先指出錯誤類型與發生的行數，引導學生思考原因，而非直接貼上修好的程式碼。
  3. **確保環境乾淨**：在 Python 開發環境中，必須引導學生使用 `uv` 建立與管理虛擬環境，嚴禁直接使用全域環境。
- **文件規範**：嚴格監督學生完成並更新專案中的四大文件：`requirements.md`、`implementation_plan.md`、`task.md` 與 `walkthrough`
```

第二步：設計三個階段性 Workflows

我們可以將開發生命週期拆解為三個好記的斜線指令：1. `/edu-start`：需求對齊與分析 2. `/edu-plan`：架構設計與任務拆解 3. `/edu-`

以下是這三個 Workflow 檔案的具體內容定義：

📁 1. edu-start.md (問題描述與需求分析)

檔案路徑：`.agents/workflows/edu-start.md` 用途：引導學生寫出需求，並以問答方式收斂與對齊目標。

```
---
description: "【階段一】引導學生完成問題描述、需求分析，並寫入 requirements.md"
---

1. 建立需求文件範本
   在專案根目錄為學生建立 `requirements.md` 檔案，包含以下結構：
   - 專案名稱與目標 (Goals)
   - 使用者故事與功能需求 (Functional Requirements)
   - 非功能需求 (如效能、資料庫選型) (Non-functional Requirements)
   - 限制條件與環境需求 (Constraints)

2. 需求引導與問答 (Alignment)
   引導學生填寫 `requirements.md`。對學生提出 2~3 個澄清問題 (例如：「這個系統需要儲存歷史資料嗎？」、「使用者會怎麼操作這個功

3. 需求確認
   引導學生閱讀並確認 `requirements.md` 的內容是否完整。確認無誤後，提醒學生輸入 `/edu-plan` 進入下一階段。
```

📁 2. edu-plan.md (計畫制定與進度追蹤)

檔案路徑：`.agents/workflows/edu-plan.md` 用途：將需求轉化為實作計畫與 TODO 列表。

```
---
description: "【階段二】引導學生制定實作計畫（implementation_plan.md）與進度追蹤表（task.md）"
---
```

- 1. 系統設計引導
讀取 `requirements.md`，引導學生思考以下設計：
 - 專案架構（例如：MVC 架構、檔案目錄結構）
 - 資料庫或資料結構設計
- 2. 制定實作計畫（Plan）
協助學生建立 `implementation\_plan.md`，內容應包含：
 - 預計建立的檔案/元件列表
 - 預計引進的套件/工具（引導使用 uv 管理）
 - 測試與驗證計畫（如何確認程式有寫對？）> [!NOTE]
> 提醒學生審查此計畫。經學生確認 OK 後，才能進行下一步。
- 3. 建立進度追蹤表（Task）
引導學生建立 `task.md`，將實作計畫拆解成多個小型的開發步驟（TODO List），例如：
 - `[ ]` 步驟一：環境初始化與依賴套件安裝
 - `[ ]` 步驟二：實作核心演算法/資料模型
 - `[ ]` 步驟三：實作使用者介面或 API
 - `[ ]` 步驟四：撰寫測試案例
- 4. 開始開發
引導學生按照 `task.md` 的步驟開始編寫程式碼。提醒學生每次完成一個步驟時，要將 `[ ]` 更新為 `[x]`。

📁 3. edu-verify.md (驗證交付與沉澱進化)

檔案路徑：.agents/workflows/edu-verify.md 用途：在程式開發告一段落時，引導學生進行驗證，並沉澱學習到的知識。

```
---
description: "【階段三】引導學生測試程式碼、撰寫 walkthrough.md，並沉澱知識與反思"
---
```

- 1. 測試與驗證（Verification）
引導學生執行 `implementation\_plan.md` 中規劃的測試方法。如果是 Python 專案，引導學生執行測試指令或手動驗證腳本。
- 2. 撰寫交付報告
協助學生在專案根目錄建立 `walkthrough.md`，內容應包含：
 - 完成的功能說明
 - 測試結果與螢幕截圖/Log
 - 開發過程中所遭遇的困難與解決方法（Reflections）
- 3. 沉澱進化（Evolution）
引導學生進行複盤：
 - 這次學到了什麼新的寫法或指令？
 - 是否有可以重複使用的程式片段或環境設定？
 - 建議學生使用 `/learn` 將有用的開發規範寫入 `AGENTS.md`，或使用 `workflow-skill-creator` 將通用流程沉澱為自訂的 Custom Skill。

如何在課堂上引導學生使用？

您可以這樣引導學生：
1. 開案時：學生打開 IDE，在對話框輸入 `/edu-start`。Agent 就會一步步帶他釐清需求、建立 `requirements.txt` 與 `implementation_plan.md`。Agent 會帶他設計架構、拆解任務，建立 `implementation_plan.md` 與 `task.md`。
3. 動手開發：學生根據 `task.md` 開始寫 Code，而 Agent 會依據 `AGENTS.md` 的引導原則（不直接給答案、只引導思考）進行輔助。
4. 開發結束後：學生輸入 `/edu-verify`。Agent 會引導他做期末複盤與知識沉澱。

您覺得這樣的模組化引導流程適合學生的學習節奏嗎？如果您同意，我可以現在直接為您專案中建立這幾個 workflow 檔案與 AGENTS

U

使用者 (User)

@[code_block] 程式碼還是交給你寫 學生負責審查和確認

A

Antigravity (Assistant)

我已將適合此「Agent 實作、學生審查與驗證」教學模式的工作流程計畫寫入 [implementation_plan.md](#) 檔案中。

核心調整重點：

- 角色互換：由 Agent 主導程式碼實作與設計，學生擔任 Reviewer 與 Approver (審查計畫、確認功能、驗證結果)。
- 四大文件串聯：
- requirements.md (需求對齊)
- implementation_plan.md (計畫制定與審查)
- task.md (任務拆解與追蹤)
- walkthrough.md (驗證交付與進化)

請點擊上方連結或介面中的「Proceed」按鈕進行審查。若符合您的期待，我將會為您在專案目錄中建立對應的規則與工作流程檔案。

U

使用者 (User)

學生開發教學引導原則

- 角色定位：你是一位有耐心的資深軟體開發助教。請使用繁體中文。
- 引導原則：
- 不要直接給出完整的程式碼：提供結構範例、虛擬碼 (Pseudo-code) 或關鍵 API 提示，引導學生自己動手寫。
- 引導學生思考而非給予解答：如果程式出錯，先指出錯誤類型與發生的行數，引導學生思考原因，而非直接貼上修好的程式碼。
- 確保環境乾淨：在 Python 開發環境中，必須引導學生使用 uv 建立與管理虛擬環境，嚴禁直接使用全域環境。
- 文件規範：嚴格監督學生完成並更新專案中的四大文件： requirements.md 、 implementation_plan.md 、 task.md 與 walkthrough.md 。

U

使用者 (User)

學生開發教學引導原則

- 角色定位：你是一位有耐心的資深軟體開發助教。請使用繁體中文。
- 引導原則：
- 不要直接給出完整的程式碼：提供結構範例、虛擬碼 (Pseudo-code) 或關鍵 API 提示，引導學生自己動手寫。
- 引導學生思考而非給予解答：如果程式出錯，先指出錯誤類型與發生的行數，引導學生思考原因，而非直接貼上修好的程式碼。
- 確保環境乾淨：在 Python 開發環境中，必須引導學生使用 uv 建立與管理虛擬環境，嚴禁直接使用全域環境。
- 文件規範：嚴格監督學生完成並更新專案中的四大文件： requirements.md 、 implementation_plan.md 、 task.md 與 walkthrough.md 。

U

使用者 (User)

程式碼還是交給你寫 學生負責審查和確認 修改一下內容

學生開發教學引導原則

- **角色定位**：你是一位有耐心的資深軟體開發助教。請使用繁體中文。
- **引導原則**：
 - **不要直接給出完整的程式碼**：提供結構範例、虛擬碼 (Pseudo-code) 或關鍵 API 提示，引導學生自己動手寫。
 - **引導學生思考而非給予解答**：如果程式出錯，先指出錯誤類型與發生的行數，引導學生思考原因，而非直接貼上修好的程式碼。
 - **確保環境乾淨**：在 Python 開發環境中，必須引導學生使用 `uv` 建立與管理虛擬環境，嚴禁直接使用全域環境。
 - **文件規範**：嚴格監督學生完成並更新專案中的四大文件：`requirements.md`、`implementation_plan.md`、`task.md` 與 `walkthrough.md`。

A

Antigravity (Assistant)

我已根據您的要求，將規則與工作流程調整為「**Agent 主導實作撰寫、學生負責審查與驗證**」的教學模式，並將所有檔案建立完成。

🔧 變更與交付成果

1. [AGENTS.md](#) (已更新規則)：
2. 角色定位修改為「資深軟體開發助教」。
3. **引導原則更新**：明確指出由 Agent (我) 負責寫 Code，但學生必須負責審查實作計畫 (`implementation_plan.md`) 並驗證最終成果。
4. 在實作過程中，我會主動向學生解釋關鍵設計決策，加強教學效果。
5. 工作流程定義檔 (已建立於專案中)：
6. [/edu-start](#)：階段一，引導學生填寫專案目標並建立 `requirements.md`。
7. [/edu-plan](#)：階段二，由 Agent 提供架構設計與 `implementation_plan.md`，學生審查批准後，Agent 初始化 `task.md` 並開始寫 Code。
8. [/edu-verify](#)：階段三，執行測試、產生 `walkthrough.md` 供學生驗證，並引導複盤進行 `/learn` 或建立 Custom Skill。

完整的交付細節與任務追蹤，已詳細記錄在 [walkthrough.md](#) 與 [task.md](#)。

💡 學生使用方式

當學生在 IDE 側邊對話欄輸入 `/` 時，即可看到三個全新的引導指令：
1. 輸入 `/edu-start` 開始收集與對齊需求。
2. 需求完成後，輸入 `/edu-plan` 由 Agent 撰寫計畫並等待學生審查批准，批准後 Agent 自動實作。
3. 實作完成後，輸入 `/edu-verify` 由 Agent 測試與交付報告，並帶領學生進行複盤。

U

使用者 (User)

列出所有檔案

A

Antigravity (Assistant)

目前專案目錄與 `.agents` 設定資料夾中的所有檔案列表如下（點擊連結即可直接開啟檔案）：

專案核心與執行檔案

- [gui_merge_tickets.py](#)：高鐵票券處理 GUI 介面程式碼。
- [merge_tickets.py](#)：高鐵票券處理核心 Python 腳本。
- [run_in_conda.bat](#)：用於 Anaconda 環境中安裝與執行的 Windows 批次檔。
- [merged_tickets.pdf](#)：合併後的高鐵票券 PDF 輸出檔。

專案開發管理文件

- [implementation_plan.md](#)：原專案 PDF 票券擷取與合併實作計畫。
- [implementation_plan_gui.md](#)：原 GUI 介面實作計畫。
- [task.md](#)：原專案任務進度追蹤。
- [task_gui.md](#)：原 GUI 任務進度追蹤。
- [walkthrough.md](#)：原專案開發驗證報告。
- [walkthrough_gui.md](#)：原 GUI 開發驗證報告。

Agent 客製化與教學設定 (`.agents/`)

這部分定義了引導學生的 Rules 與 Workflows：

- [AGENTS.md](#)：專案規則與學生開發教學引導原則 (Rules)，規定 Agent 寫 Code 與學生 Review 的機制。

📄 工作流程 (Workflows) — `.agents/workflows/`

- [edu-start.md](#)：階段一的工作流程檔案（對應指令 `/edu-start`）。
- [edu-plan.md](#)：階段二的工作流程檔案（對應指令 `/edu-plan`）。
- [edu-verify.md](#)：階段三的工作流程檔案（對應指令 `/edu-verify`）。

🛠️ 自訂技能 (Skills) — `.agents/skills/`

- [SKILL.md](#)：高鐵票券處理的自訂技能說明文件。

U

使用者 (User)

學生開發教學引導原則

- **角色定位**：你是一位有耐心的資深軟體開發助教。請使用繁體中文。
- **引導原則**：
 - **Agent 實作與編寫**：所有的程式碼均由 Agent (你) 負責撰寫與實作。但必須在實作前提供計畫供學生審查，並在實作後由學生確認與驗證。
 - **引導學生審查與思考**：在撰寫程式碼之前，必須先撰寫實作計畫 (`implementation_plan.md`) 提交給學生審查。在實作或除錯過程中，主動解釋關鍵設計決策，幫助學生學習程式架構與邏輯。
 - **確保環境乾淨**：在 Python 開發環境中，引導學生使用 `uv` 建立與管理虛擬環境，嚴禁直接使用全域環境。
 - **文件規範**：嚴格監督並引導學生審查、確認並更新專案中的四大文件：`requirements.md`、`implementation_plan.md`、`task.md` 與 `walkthrough.md`。

以上內容並沒有提到 引導學生描述問題與需求

A

Antigravity (Assistant)

我已將「問題與需求引導」原則成功加入 `.agents/AGENTS.md` 規則中，使整個引導生命週期更加完整。



已更新的 [AGENTS.md](#) 規則細節：

在「引導原則」第一點明確加入了：

1. 問題與需求引導：主動引導學生描述他們碰到的問題與開發需求，並由 Agent (你) 整理撰寫成 `requirements.md` 文件，透過互動問答收斂與對齊細節。

這項規則更新也已同步記載於交付報告 [walkthrough.md](#) 中。

現在，只要輸入 `/edu-start`，Agent 就會遵循此新規則，從引導學生描述問題開始，逐步產出 `requirements.md` 並展開後續流程。

U

使用者 (User)

請將我的問題和你的回應 這些對話紀錄存成pdf檔案

A

Antigravity (Assistant)

Task is running in the background. I will wait for it to complete.